

UNIVERZA NA PRIMORSKEM
FAKULTETA ZA MATEMATIKO, NARAVOSLOVJE IN
INFORMACIJSKE TEHNOLOGIJE

ZAKLJUČNA NALOGA
(FINAL PROJECT PAPER)

**VREDNOTENJE PRENOSLJIVOSTI MODELOV
STROJNEGA UČENJA ZA ZAZNAVANJE DDOS
NAPADOV
(EVALUATING TRANSFERABILITY OF
MACHINE LEARNING MODELS FOR DDOS
DETECTION)**

DANILO LAZIĆ

UNIVERZA NA PRIMORSKEM
FAKULTETA ZA MATEMATIKO, NARAVOSLOVJE IN
INFORMACIJSKE TEHNOLOGIJE

Zaključna naloga

(Final project paper)

**Vrednotenje prenosljivosti modelov strojnega učenja za
zaznavanje DDoS napadov**

(Evaluating transferability of machine learning models for DDoS detection)

Ime in priimek: Danilo Lazić

Študijski program: Računalništvo in informatika

Mentor: doc. dr. Aleksandar Tošić

Somentor: asist. Domen Vake

Koper, avgust 2026

Ključna dokumentacijska informacija

Ime in PRIIMEK: Danilo LAZIĆ

Naslov zaključne naloge: Vrednotenje prenosljivosti modelov strojnega učenja za zaznavanje DDoS napadov

Kraj: Koper

Leto: 2026

Število listov: 32

Število slik: 4

Število referenc: 27

Število tabel: 5

Mentor: doc. dr. Aleksandar Tošić

Somentor: asist. Domen Vake

Ključne besede: Network intrusion detection, machine learning, cross-domain evaluation, generalisation gap, benchmark datasets, real-world traffic, majority-class collapse, DDoS

Izvleček: V zadnjih dveh desetletjih so bili sistemi za zaznavanje vdorov, ki temeljijo na machine learning in deep learning, predmet obsežnih raziskav, pri čemer številne študije poročajo o skoraj popolni klasifikacijski natančnosti na benchmark datasetih, kot so NSL-KDD, KDD99 in družina CIC-IDS. Ta zaključna naloga preučuje, ali takšni rezultati odražajo dejanske zmožnosti zaznavanja napadov ali pa so posledica overfittinga. Ovrednotenih je bilo trinajst open-source ML-NIDS implementacij, ki zajemajo Random Forest, SVM, Decision Trees ter CNN-LSTM in GRU modele. Vsak model je bil ponovno ovrednoten na svojem benchmark datasetu za vzpostavitev reproducibility baseline. Za ocenjevanje je bil pripravljen evaluation dataset iz zajemov dejanskega omrežnega prometa z 1,28 milijona omrežnih tokov (flows), zbranih aprila 2026. Podatki so bili obdelani v CIC in NSL-KDD feature sete z uporabo lastnega postopka obdelave PCAP datotek. Pripravljene so tri različice dataseta: heuristic baseline, Suricata-based in botnet-injected s 400.000 sintetično generiranimi malicious tokovi, ki zagotavljajo preverjen ground truth. Evalvacija na botnet-injected različici je razkrila izrazito vrzel v posploševanju: enajst od trinajstih modelov je doseglo rezultate na ravni ali pod majority-class baseline (0,7629), večina pa je pokazala popoln majority-class collapse in skoraj ničelni malicious recall. Rezultati kažejo, da dominantni ML-NIDS benchmark dataseti ne predstavljajo več dejanskih pogojev operativnih omrežij, zato lahko temeljne pomanjkljivosti pri posploševanju modelov ostanejo neopažene.

Key words documentation

Name and SURNAME: Danilo LAZIĆ

Title of final project paper: Evaluating transferability of machine learning models for DDoS detection

Place: Koper

Year: 2026

Number of pages: 32

Number of figures: 4

Number of references: 27

Number of tables: 5

Mentor: Assoc. Prof. Aleksandar Tošić, PhD

Co-Mentor: Assist. Domen Vake

Keywords: Network intrusion detection, machine learning, cross-domain evaluation, generalisation gap, benchmark datasets, real-world traffic, majority-class collapse, DDoS

Abstract: Over the past two decades, machine learning and deep learning based intrusion detection systems have been extensively researched, with studies reporting near-perfect classification accuracy on benchmark datasets such as NSL-KDD, KDD99, and the CIC-IDS family. This thesis examines if such results are genuine detection capability, are artifacts of dataset overfitting or both. Thirteen open-source Machine Learning Network Intrusion Detection System implementations were evaluated, spanning classical approaches such as Random Forest, Support Vector Machine and Decision Trees, as well as deep learning architectures such as Convolutional Neural Network - Long Short-Term Memory and Gater Recurrent Unit based models, each first re-evaluated on its native benchmark dataset to establish a reproducibility baseline. A real-world evaluation dataset was constructed from live network traffic captures totalling 1.28 million flows in April 2026, processed into CIC and NSL-KDD style flow feature sets using a custom PCAP parsing pipeline, with three labeling variants: a heuristic baseline, a Suricata-based variant and a botnet-injected variant with 400,000 synthetically generated malicious flows providing verified ground truth. Evaluation on the botnet variant revealed a consistent generalization gap: eleven of the thirteen models performed at or below the majority-class baseline of 0.7629, with most exhibiting complete majority-class collapse and near-zero malicious recall. These findings suggest that dominant intrusion detection system benchmarks have become decoupled from operational network conditions, allowing fundamental generalization failures to go unnoticed.

Acknowledgement

Firstly, I am greatly grateful to my mentor and co-mentor for guiding me through the whole process of writing this thesis. Their support, responsiveness, and willingness to answer every question I had, greatly helped me complete this thesis.

I would also like to express my great gratitude to my mother, who provided me with support and belief, as well as my sister and brother-in-law. I am grateful that they provided me with the confidence and motivation to overcome any challenge I faced on this academic journey.

I am also grateful to everyone who contributed, directly or indirectly, to the completion of this work.

Contents

1	Introduction	1
2	Background	3
3	Implementations and methodology	8
4	Dataset Construction	15
4.1	Feature Extraction Pipeline	16
4.2	Heuristic Labeling	16
4.3	Suricata-Based Labeling	17
4.4	Botnet-Injected Variant	17
4.5	CIC Supplementary Dataset	18
5	Cross-domain evaluation	19
5.1	Evaluation Methodology	19
5.2	Overview of Results	20
5.3	CIC-Family Models	23
5.4	NSL-KDD-Family Models	24
5.5	Summary	25
6	Implications	27
7	Conclusion	31
8	Bibliography	33

List of Tables

1	Overview of commonly used NIDS benchmark datasets	6
2	Overview of evaluated ML-NIDS implementations	9
3	Reported vs. reproduced native accuracy per implementation	10
4	Overview of constructed real-world evaluation dataset variants	15
5	Cross-domain evaluation results per model and dataset variant	20

List of Figures

1	Reported accuracy of evaluated implementations by publication year	12
2	Reported vs. reproduced native accuracy per implementation, grouped by architecture family	13
3	Native (reproduced) vs. cross-domain accuracy per implementation on the botnet-injected dataset	21
4	Per-class precision and recall on the botnet-injected dataset, grouped by training dataset family	22

List of Abbreviations

BGRU	Bidirectional Gated Recurrent Unit
CIC	Canadian Institute for Cybersecurity
CNN	Convolutional Neural Network
DARPA	Defense Advanced Research Projects Agency
DBN	Deep Belief Network
DDoS	Distributed Denial of Service
DNS	Domain Name System
DNN	Deep Neural Network
DoS	Denial of Service
FTP	File Transfer Protocol
GRU	Gated Recurrent Unit
HTTP	Hypertext Transfer Protocol
IDS	Intrusion Detection System
IP	Internet Protocol
LSTM	Long Short-Term Memory
ML	Machine Learning
MLP	Multilayer Perceptron
NIDS	Network Intrusion Detection System
PCAP	Packet Capture
POP3	Post Office Protocol 3
R2L	Remote to Local
RF	Random Forest
SMTP	Simple Mail Transfer Protocol
STL	Self-Taught Learning
SVM	Support Vector Machine
TCP	Transmission Control Protocol
TF	TensorFlow
U2R	User to Root
UDP	User Datagram Protocol
XGBoost	Extreme Gradient Boosting

1 Introduction

Computer networks carry considerable economic value, such as banking transactions, medical records, industrial control signals, ordinary web browsing, which makes them appealing targets for cyber attacks. Among the various types of attacks, one of the most disruptive ones is the Distributed Denial of Service (DDoS) attack, where an attacker overwhelms a target server or network with a massive volume of traffic, often generated by hundreds or thousands of compromised devices centrally coordinated and commonly referred to as a botnet, with the goal of exhausting its resources and making the service unavailable for users. DDoS attacks are relatively cost effective to launch, hard to trace back to a single source and can take down anything from a small website to critical infrastructure, which is why detecting them quickly and reliably is important.

For a long time, intrusion detection relied on signature-based systems: tools that compare traffic against a database of known attack patterns, similar to how antivirus software recognizes known malware. A DDoS attack, however, is not defined by a fixed signature as much as by scale and pattern (many sources, coordinated timing, abnormal traffic volume), which makes it a natural target for statistical, learning-based detection instead. Machine learning offered such an alternative. Rather than matching known patterns, a model could be trained to detect the general statistical shape of a traffic flood, how packets are timed, how large flows are, how many sources are involved, and distinguish an attack it had never encountered before from benign traffic.

At first glance, the problem of network intrusion detection appears to be largely solved. Over the past two decades, machine learning based approaches have been an extensively studied topic, with published results regularly exceeding 99% classification accuracy on datasets such as NSL-KDD [1], KDD99 [2], the CIC-IDS [3–5] family and UNSW-NB15 [6]. Despite all these impressive numbers, real-world adoption of Machine Learning (ML) based detection systems remains surprisingly limited and most operational security teams still depend on traditional signature-based systems, such as Suricata. This gap between reported performance and practical deployment raises a fundamental question: do these benchmark results reflect genuine detection capability, are they merely artifacts of overfitting to datasets that no longer represent modern

network traffic, or something else entirely?

This thesis investigates that question directly. The focus is placed on evaluation, specifically on whether existing open-source machine learning network intrusion detection system (ML-NIDS) implementations perform well when tested against real-world network traffic. Thirteen open-source implementations were selected, covering this range of classical and deep learning approaches. Each model was first re-evaluated on its original training dataset to verify the results reported in its source publication, establishing a reproducibility baseline.

The contributions of this work are the re-implementation and testing of existing models, merging compatible datasets, as well as the construction and application of a real-world evaluation dataset derived from live network traffic captures totaling approximately 1.68 million flows, collected over three periods in April 2026. This dataset was processed into both CIC-style and NSL-KDD-style flow feature sets using a custom PCAP parsing pipeline, and ground truth labels were supplemented using Suricata, an open-source intrusion detection engine, to identify high-confidence malicious flows. This makes it possible to compare each model's reported accuracy directly against its accuracy on the real-world captures, model by model.

The remainder of this thesis is structured as follows. Chapter 2 provides background on network intrusion detection and the datasets most commonly used in the field. Chapter 3 describes the evaluated implementations and the reproducibility methodology. Chapter 4 details the construction of the real-world evaluation dataset. Chapter 5 presents the cross-domain evaluation results and analysis. Chapter 6 discusses the implications of these findings. Chapter 7 concludes the thesis with a summary of findings, limitations, and directions for future work.

2 Background

As networks increase in complexity and scale, the threat landscape facing modern organizations has also expanded. Cyberattacks ranging from distributed denial of service (DDoS) floods to stealthy data exfiltration (the covert theft of sensitive data from a network) have become a lot more frequent and sophisticated, placing significant pressure on security infrastructure. Network intrusion detection systems (NIDS) are an important line of defense, with them being tasked with identifying malicious traffic in real time by closely monitoring network flows and flagging anomalous or known malicious patterns. Traditionally, such systems relied on manually crafted signatures (rules designed by security experts) to match known attack patterns. While effective against well-documented threats, signature-based approaches struggle to generalize to novel or zero-day attacks, motivating the research community to explore machine learning as a more adaptive alternative.

The idea of using machine learning and deep learning for intrusion detection is not novel. It gained attention in the late 1990s, one of the reasons being the KDD (Knowledge Discovery in Data) Cup 1999 competition, which gave academics a common dataset to work with and compare results against [1]. KDD99 drew attention for research on data-driven classification by making it easier to compare learned models alongside established signature-based, rule-based and anomaly-based detection approaches. Over the following two decades, the field grew rapidly. Decision trees, support vector machines and later deep learning architectures such as convolutional neural networks were all applied to the problem, with them reporting very high accuracy figures. Results above 99% became almost routine in the literature. These figures could give the impression of near-solved benchmark classification, even though operational intrusion detection remained substantially more difficult.

Beyond the datasets themselves, it is worth tracing how the specific machine learning approaches evaluated later in this thesis developed over time, since much of the field's history is really a sequence of authors matching, and then narrowly surpassing each other's benchmark numbers.

Niyaz et al. [7] were among the earliest, introducing a self-taught learning approach that pretrains a sparse autoencoder on unlabeled NSL-KDD traffic before attaching a softmax classifier, and showed that this two-phase architecture could classify NSL-KDD test traffic with 88.39% binary accuracy, one of the first widely cited deep learning treatments of the problem. Nkima et al. [8] approached the problem from the feature side rather than the model side, showing that a comparatively simple decision tree combined with recursive feature elimination could reach up to 99.95% class accuracy on the harder U2R attack category of NSL-KDD.

Vinayakumar et al. then published series of three related studies in 2017 and 2019 that systematically compared entire architecture families against each other, applying convolutional neural networks [9], deep belief networks and other shallow-versus-deep hybrids [10], and RNN (Recurrent Neural Network), LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) variants [11]. Combined into a single implementation for this thesis's evaluation, these three studies reported 97.30% accuracy on NSL-KDD. Taken together, they established that competitive NSL-KDD performance was achievable across essentially every architecture family available at the time, not just one.

The following year, Xu et al. [12] combined a bidirectional GRU with an MLP (Multilayer Perceptron) classifier and showed that this recurrent design could outperform the feedforward and convolutional models that preceded it, reporting 99.24% on NSL-KDD dataset, among the highest figures reported for a recurrent architecture on this dataset. Basnet et al. [13] moved the comparison to the newer CIC-IDS2018 dataset in 2019, showing that deep neural networks implemented across three different frameworks (fastai, Keras and TensorFlow) converged on compatible accuracy around 99% regardless of implementation, while Perez and Danner [14], working the same year, took an unsupervised route instead, showing that clustering algorithms such as K-Means could still separate attack and benign traffic without labels, though at a lower 96.19% accuracy, a gap that reflects the added difficulty of the unsupervised setting rather than a weaker implementation.

Agarwal et al. [15] returned to a supervised ensemble approach in 2020, combining KNN (K-Nearest Neighbors), a CNN-LSTM hybrid and Random Forest into a three-stage voting system, and showed that this ensemble could reach 97.83% accuracy on KDD99. The following year, Assis et al. [16] applied a GRU architecture to the newer CIC-DDoS2019 and CIC-IDS2018 datasets specifically for DDoS detection in software-defined networks, reporting 99.94% and 97.09% accuracy respectively, while Serinelli

et al. [17] compared SVM (Support Vector Machine), Random Forest and XGBoost (Extreme Gradient Boosting) directly against each other on both KDD99 and NSL-KDD, and showed that Random Forest and SVM in particular could reach a 99.60% and 94.44% accuracy on NSL-KDD. Finally, Halladay et al. [18], publishing in 2022, benchmarked nine classical and deep learning models side by side on a custom time-based feature set and showed that several of them, including a comparatively simple Linear Discriminant classifier, could match the ensemble and deep learning approaches that preceded it at 99.99% binary accuracy.

Read together, these studies describe a field that kept finding new routes to the same ceiling: whether through recurrent architectures, ensembles, richer engineered features or entirely different algorithm families, accuracy in the high nineties on KDD99, NSL-KDD or a CIC-IDS variant became the expected outcome of a new paper rather than a noteworthy result. The question few of these studies address directly is whether that ceiling says more about the models or about the benchmarks they were all measured against.

The datasets that are commonly used across the NIDS research papers each have their own origin story. The KDD Cup '99 dataset, which was created from the 1998 DARPA intrusion detection evaluation program, was the de facto standard for benchmarking intrusion detection systems for a long time. It consists of around 4.9 million connection records, labeling 23 attack categories, consisting of DoS, Probe, R2L (Remote to Local), and U2R (User to Root) families. Despite the widespread use in research, the dataset has been heavily criticized over the years for having a large number of duplicate records [1]. The high level of redundancy can bias models toward frequently repeated patterns and inflate aggregate performance, while obscuring poor detection of rare or previously unseen attacks. NSL-KDD (Network Security Laboratory and Knowledge Discovery and Data Mining) was later introduced in 2009 as a way to overcome these problems, removing the duplicate records and reducing some of the severe frequency biases in KDD99, although substantial class imbalance remained. While this addressed some of the most obvious flaws of its predecessor, NSL-KDD still reflects network traffic from the late 1990s, a period when the internet and the attacks targeting it looked very different from what is seen today.

The Canadian Institute for Cybersecurity (CIC) addressed some of these concerns by releasing a series of more modern datasets throughout the late 2010s. The CIC-IDS2017 dataset was among the first of these, generated in a controlled lab environment using realistic background traffic and a variety of attack scenarios including brute force,

DDoS and web attacks. It introduced a richer set of flow-based features extracted using the CICFlowMeter [19] tool, moving away from the connection-level features that characterized KDD99 and NSL-KDD. The CIC family didn't stop there. CIC-IDS2018 and CIC-DDoS2019 followed, each bringing additional attack scenarios and broader coverage. Around the same time, researchers at the University of New South Wales took a different approach with UNSW-NB15, mixing real modern background traffic with synthetically generated attacks across nine categories using the IXIA PerfectStorm¹ tool. Both efforts pushed the field forward in meaningful ways, but the core limitation remained, which was that all of it was still generated in a lab, and lab traffic only goes so far in representing what a real network actually looks like.

Table 1: Overview of commonly used NIDS benchmark datasets

Dataset	Year	Samples	Features	Attack Families
KDD99	1999	4,898,431	41	DoS, Probe, R2L, U2R
NSL-KDD	2009	148,517	41	DoS, Probe, R2L, U2R
CIC-IDS2017	2017	2,830,743	79	DoS/DDoS, Brute Force, Botnet
CIC-IDS2018	2018	16,232,943	80	DoS/DDoS, Brute Force, Botnet
CIC-DDoS2019	2019	50,063,112	88	Reflection & Exploitation DDoS
UNSW-NB15	2015	2,540,044	49	DoS, Exploits, Fuzzers, Recon

Despite their differences in origin and design, all of these datasets share a fundamental problem, which is that they were generated in controlled, artificial environments rather than captured from live operational networks. The traffic patterns, attack distributions and even the timing of flows are artifacts of how each dataset was constructed, not reflections of what network traffic actually looks like in production. This matters because machine learning models are highly sensitive to the statistical properties of the data they are trained on. A model trained on KDD99 or NSL-KDD learns to distinguish malicious from benign traffic based on patterns specific to late 1990s network behavior. Likewise, models trained on CIC or UNSW-NB15 datasets learn distributions that reflect the simulated setup rather than a real organization's network. When these models encounter traffic from a different environment, those patterns may no longer hold, causing the performance to degrade in ways that benchmark evaluations would never reveal.

What makes this situation particularly concerning is that the limitations of these datasets have not gone unnoticed. Researchers have been pointing out the flaws of KDD99 since at least the mid-2000s [1], and the issues with newer datasets have been

¹IXIA PerfectStorm is a commercial network testing platform.

documented as well [20,21]. Yet despite this awareness, the same benchmarks continued to be used as the primary means of evaluating new models, and reported accuracy figures kept climbing. Part of the reason for this is that using an established dataset makes it easier to compare results across papers, and collecting real network traffic comes with its own set of challenges around privacy, labeling and infrastructure. But the side effect is that the field has developed a tendency to optimize for benchmark performance rather than real-world effectiveness. A model that scores 99.8% [12,17] on NSL-KDD is not necessarily a model that would perform well on a live network, and the gap between those two things is exactly what this thesis sets out to measure.

3 Implementations and methodology

This chapter describes the thirteen open-source ML-NIDS implementations selected for evaluation, the methodology used to reproduce their originally reported results and the process of evaluating them against real-world network traffic. The goal was not to propose new models or improve upon existing ones, but to assess whether the performance figures reported in the source publications could be reproduced, and more importantly, whether those models retained any meaningful detection capability when exposed to traffic outside their native benchmark environment.

The selection process focused on finding open-source implementations that were directly tied to a published research paper and reproducible without requiring access to proprietary infrastructure or licensed software. Starting from a pool of seventeen candidate implementations, four were excluded before evaluation began. One implementation lacked any published paper altogether and consisted only of public code without an accompanying research contribution. A second consisted entirely of a dataset preprocessing pipeline with no novel algorithm or model of its own, offering no meaningful basis for performance evaluation. A third implementation, which was a fuzzy logic based intrusion detection system, had its public GitHub repository deleted prior to evaluation, making reproducibility impossible regardless of its academic merit. A fourth, a PyTorch-based neural network voting system, was never successfully integrated into the evaluation pipeline due to compatibility issues with the orchestration framework. The remaining fourteen implementations were drawn from eleven distinct repositories, with some repositories containing multiple model variants. Specifically, one repository contributed two implementations using different deep learning frameworks, which are FastAI and Keras with TensorFlow, while another contributed three classical machine learning variants using Random Forest, SVM and XGBoost respectively. A further three papers sharing the same Vinayakumar et al. authorship were hosted in a single repository and were treated as one combined implementation. It is worth noting that the dataset choices were not a selection criterion, it simply turned out that the vast majority of implementations naturally leaned toward the same

two dataset families, NSL-KDD and CIC-IDS collection, which are by far the most commonly used benchmarks in this area of research.

Table 2: Overview of evaluated ML-NIDS implementations

Model Alias	Architecture	Dataset	Citation
deeplearning_ids_fastai	Deep Learning (FastAI)	CIC-IDS2018	[13]
deeplearning_ids_keras_tf	Deep Learning (Keras/TF)	CIC-IDS2018	[13]
ddos_traffic_research_notebooks	Deep Learning	CIC-DDoS2019	[18]
intrusion_detection_kdd_nsl	GRU-based DNN	NSL-KDD	[7]
cic_ddos2019_deep_learning	GRU Deep Learning	CIC-DDoS2019	[16]
network_intrusion_detection_bulk	MLP/DBN/CNN	NSL-KDD	[9–11]
decisiontree_ids	Decision Tree	NSL-KDD	[8]
cbad_clustering	Clustering (K-Means)	NSL-KDD	[14]
nids_python_eval_rf	Random Forest	NSL-KDD	[17]
nids_python_eval_svm	SVM	NSL-KDD	[17]
nids_python_eval_xgboost	XGBoost	NSL-KDD	[17]
nsl_gru_runner	GRU	NSL-KDD	[12]
anomaly_detection_kdd99_cnnlstm	CNN-LSTM	KDD99	[15]

Each implementation was first executed on its native dataset, which is the same dataset that the original authors used to train and evaluate their model. This step served as a reproducibility check: an implementation was considered to have reproduced its reported result if its accuracy fell within approximately 5 percentage points of the original figure, with deviations beyond this treated as a signal warranting closer inspection rather than automatic disqualification. It is worth noting that this margin has inherent limits, that most of the reviewed papers did not report full hyperparameter settings, exact train-test splits, or random seeds, so a reproduction within this margin cannot be taken as evidence that an implementation is identical to the original in every aspect. Rather, it represents the closest approximation of faithfulness that could reasonably be established given the information available in the source publications. In practice, most models reproduced their reported results closely, with native accuracy figures consistently landing around 0.99 across both NSL-KDD and CIC-family implementations. Where minor discrepancies existed, these were attributed to differences in random seeds, train-test split strategies, or minor preprocessing variations rather than fundamental implementation errors.

To ensure a consistent and comparable evaluation across all fourteen implementations, a unified orchestration framework was developed. Rather than running each repository in isolation using its own pipeline, a central runner script read from a model

and dataset registry, then dispatched training and evaluation jobs with standardised inputs and collected outputs in a common format. The design preserved the original repository structure of each implementation while allowing cross-dataset evaluation to be applied uniformly. Implementations fell into two broad categories based on how much intervention was required. Eleven models could not be run directly from their original repositories due to dependency issues, notebook-only formats, or insufficient parametrisation, so they were instead adapted into custom runner scripts that reimplemented the core architecture while keeping the original repository as reference. The remaining three, all from the same repository, had self-contained native training scripts and were run with minimal modification using their original entry points. One additional model evaluated pre-saved FastAI learner files already present in the repository, requiring no retraining at all.

Table 3: Reported vs. reproduced native accuracy per implementation

Model Alias	Dataset	Reported Acc.	Reproduced Acc.
deeplearning_ids_fastai	CIC-IDS2018	0.9868	0.9999
deeplearning_ids_keras_tf	CIC-IDS2018	0.9868	0.9999
ddos_traffic_research_notebooks	CIC-DDoS2019	0.9999	— ^a
intrusion_detection_kdd_nsl	NSL-KDD	0.8839	0.7088
cic_ddos2019_deep_learning	CIC-DDoS2019	0.9994	0.9999
network_intrusion_detection_bulk	NSL-KDD	0.9730	0.9992
decisiontree_ids	NSL-KDD	— ^b	0.9944
cbad_clustering	NSL-KDD	0.9619	0.8507
nids_python_eval_rf	NSL-KDD	0.9960	0.9944
nids_python_eval_svm	NSL-KDD	0.9444	0.9992
nids_python_eval_xgboost	NSL-KDD	— ^c	0.9988
nsl_gru_runner	NSL-KDD	0.9924	0.7489
anomaly_detection_kdd99_cnnlstm	KDD99	0.9783	0.9807

^a Pre-saved model weights were evaluated directly; no retraining was performed.

^b Paper reports per-class or per-scenario figures only; no single comparable accuracy is stated.

^c Architecture not evaluated on NSL-KDD in the source paper.

The results of the reproducibility are summarised in Table 3. For eight of the fourteen implementations, faithful replication produced accuracy figures consistently at or above 0.99 on their native dataset, broadly confirming that the original results were reproducible within reasonable margins. Four implementations warrant specific discussion.

`cbad_clustering` (Perez and Danner) reproduced at 0.8507 against a reported F-score of 0.9619, a gap consistent with the known sensitivity of K-Means clustering to

initialisation and feature scaling across different random seeds and dataset splits.

`ns1_gru_runner` (Xu et al.) reproduced at 0.7489 on the NSL-KDD test set (`KDDTest+.txt`), compared to the paper’s reported 0.9924. The gap is attributable to the deliberate difficulty of the NSL-KDD test partition, which contains novel attack subtypes absent from the training split and is known to produce substantially lower accuracy than cross-validation on the training set alone. The paper’s figure was likely measured under the more favourable training-set evaluation conditions rather than on `KDDTest+`.

`intrusion_detection_kdd_ns1` (Niyaz et al.) uses a two-phase self-taught learning architecture: a sparse autoencoder for unsupervised feature learning followed by a softmax classifier, trained on the 20% NSL-KDD subset [7]. Evaluated on `KDDTest+.txt`, it reproduced at 0.7088 against the paper’s reported 0.8839. The gap is attributable to the same structural challenge as `ns1_gru_runner`: the `KDDTest+` partition introduces 16 novel attack subtypes absent from the training split, which the STL architecture has no prior exposure to. The paper’s figure was obtained under internal cross-validation conditions on the training set, where no novel attack types are present.

`anomaly_detection_kdd99_cnnlstm` (Agarwal et al.) reproduced at 0.9799 against the reported 0.9783, marginally exceeding the paper’s figure, which is consistent with minor differences in random seed and train-test split.

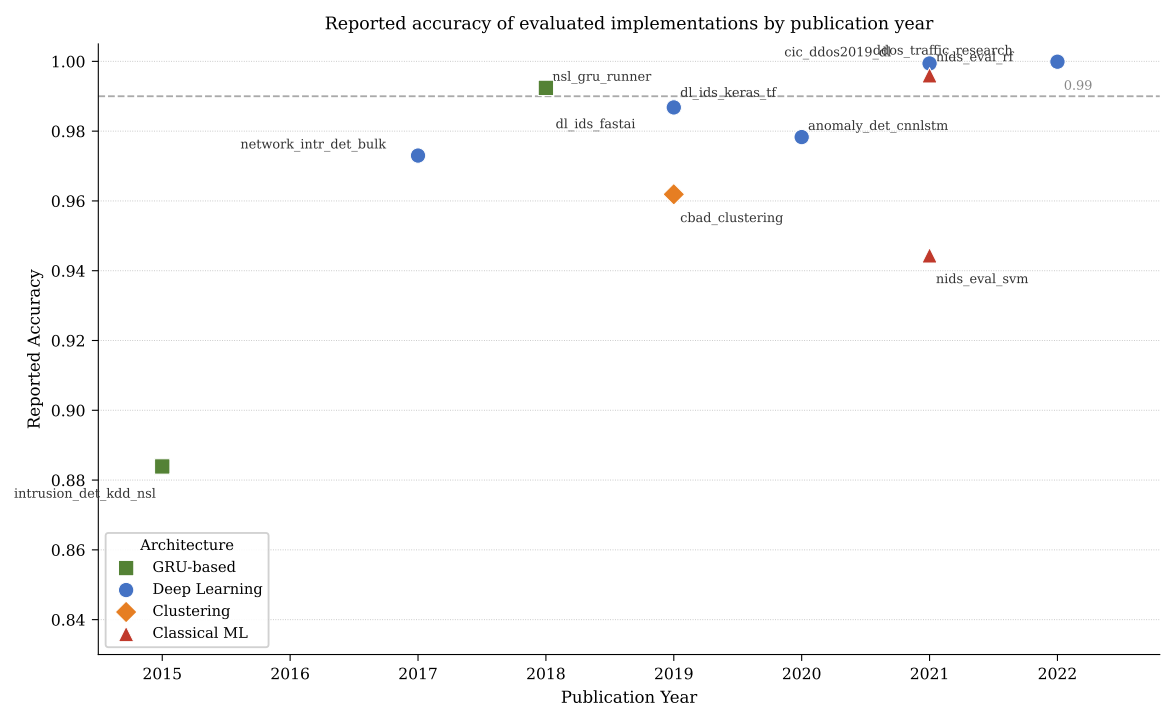


Figure 1: Reported accuracy of evaluated implementations by publication year

Each point represents one implementation; shape and color indicate architecture family. Three implementations with no single comparable reported accuracy are excluded.

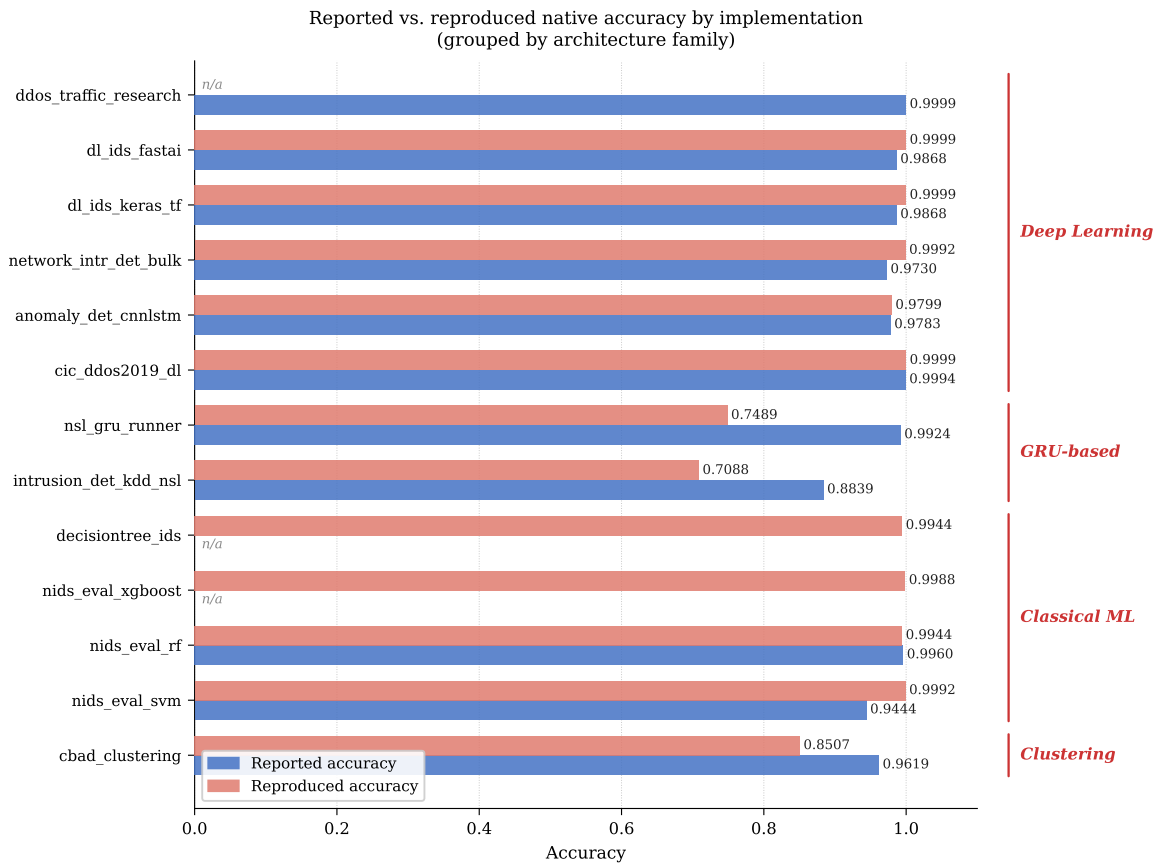


Figure 2: Reported vs. reproduced native accuracy per implementation, grouped by architecture family

Entries marked n/a indicate cases where the source paper reported no directly comparable single-metric accuracy on the native dataset, or where pre-saved model weights were evaluated without retraining.

Figure 1 illustrates the distribution of reported accuracy across publication years. The cluster of values at or above 0.99 from 2018 onward reflects the general trend in the literature toward increasingly confident benchmark claims. Two notable outliers sit below this cluster: *intrusion_detection_nsl_kdd*, whose reported figure of 0.8839 reflects the STL architecture’s inherent difficulty on held-out NSL-KDD test partition, and *nids_python_eval_svm*, which reported a lower figure due to the inherent limitations of SVM scaling on larger feature sets. Figure 2 shows that while most implementations reproduced their benchmark results closely, two models reproduced substantially below their reported figures: *ns_l_gru_runner* (0.7489 vs. 0.9924 reported), and *cbad_clustering* (0.8507 vs. 0.9619 reported), each for distinct architectural reasons discussed above. Having established this reproducibility baseline, the following chapter describes the construction of the real-world evaluation dataset against which these models will next

be tested.

Unlike the benchmark datasets described in Chapter 2, which were constructed in controlled environments with known attack distribution, the real-world dataset introduced in the following chapter was captured from live network traffic with no prior knowledge of what malicious activity, if any, would be present. This distinction is central to the evaluation design, as it is precisely this kind of uncertainty that benchmark-trained models have never been required to handle.

4 Dataset Construction

This chapter describes the construction of the real-world evaluation dataset used to assess model performance outside native benchmark conditions. Three distinct dataset variants were produced from the same source captures, each differing in how malicious traffic was identified or introduced. The traffic base consists of 1,287,049 flows extracted from three passive network captures collected in April 2026 (`out_14042026.pcap`, `out_17042026.pcap` and `out_18042026.pcap`), totalling approximately 1.68 GB of raw packet data. These captures were obtained passively from a live network and represent genuine operational traffic with no predetermined attack content. All three variants were produced in parallel feature representations: a CIC-style schema compatible with CIC-IDS family models using CICFlowMeter, and an NSL-KDD-style schema compatible with NSL-KDD trained models, resulting in six dataset files in total.

Table 4: Overview of constructed real-world evaluation dataset variants

Variant	Schema	Total flows	Benign	Malicious	Mal%
Heuristic	CIC-style	1,287,049	1,222,342	64,707	5.03%
Heuristic	NSL-KDD-style	1,287,049	1,222,342	64,707	5.03%
Suricata	CIC-style	1,287,049	1,287,033	16	<0.01%
Suricata	NSL-KDD-style	1,287,049	1,287,033	16	<0.01%
Botnet-injected	CIC-style	1,687,049	1,287,049	400,000	23.71%
Botnet-injected	NSL-KDD-style	1,687,049	1,287,049	400,000	23.71%

The decision to produce both representations in parallel, rather than selecting a single feature schema, was deliberate. NSL-KDD-trained and CIC-trained models operate on fundamentally different feature spaces and cannot be evaluated against the same dataset file without retraining or feature transformation. Producing both schemas from the same underlying flow records ensures that every model is evaluated against data derived from identical network traffic, differing only in how the traffic is represented. This matters for interpretability: any performance difference observed between a CIC-trained and an NSL-KDD-trained model on the real-world data reflects differences in the model and its training distribution, not differences in the underlying traffic they were shown. It also introduces an inherent limitation for the NSL-KDD schema specifically. Because NSL-KDD’s 41 features include host-based and content-level statistics that require deep packet inspection or sustained connection tracking to compute, and

passive flow capture provides neither, 29 of the 41 features in the NSL-KDD representation could not be populated and were zero-padded. All NSL-KDD-trained models in this evaluation therefore operate on a partially imputed feature vector, a constraint that is consistent across all NSL-KDD evaluations and constitutes a structural disadvantage relative to native benchmark conditions.

4.1 Feature Extraction Pipeline

The raw PCAP files were processed into flow-level records by using a custom parsing pipeline built on Scapy [22]. Each file was parsed packet by packet, with flows identified by their bidirectional 5-tuple (source IP, destination IP, source port, destination port, protocol). Packets that were identified as the same flow were aggregated in both directions, resulting in per-flow statistics including total packet and byte counts, forward and backward packet length means, flow duration, flow-level byte and packet rates, and TCP flag counts (SYN, ACK, FIN, RST, PSH, URG). These features were assembled into a CIC-style schema modelled on the CICFlowMeter feature set used in the CIC-IDS dataset family, resulting in 22 flow-level features per record.

A second representation was derived from the CIC-style features via a column mapping procedure implemented in `build_realworld_datasets.py`. The NSL-KDD schema requires 41 features, of which 11 could be directly mapped or approximated from the available flow statistics: `duration`, `protocol_type`, `service`, `flag`, `src_bytes`, `dst_bytes`, `land`, `wrong_fragment`, `urgent`, `count` and `srv_count`. The remaining 30 features, which capture connection-level content and host-based statistics not derivable from passive flow observation, were zero-padded. This feature mismatch is an inherent limitation of applying NSL-KDD-trained models to passively captured traffic, and is explicitly acknowledged in the cross-domain evaluation reported in Chapter 5.

4.2 Heuristic Labeling

The first variant applies a *heuristic labeling* scheme derived from flow-level statistics. Flows were ranked by a composite score weighting packet rate (50%), byte rate (30%) and SYN flag count (20%), and the top 5% by this score were labeled as `malicious_heuristic`, yielding 64,707 malicious flows (5.03%). This approach does not constitute ground truth and is explicitly acknowledged as a pseudo-labeling strategy; its purpose is to provide a stress-test signal under conditions where no verified attack traffic is present, and its results should be interpreted accordingly.

4.3 Suricata-Based Labeling

The second variant uses *Suricata-based labeling*. Suricata [23], an open-source rule-based intrusion detection engine, was applied to the same PCAP captures in EVE JSON output mode. High-severity (severity 1) alerts were extracted and matched to flows by 5-tuple. This process identified only 16 unique malicious flows across the entire 1,287,049-flow dataset, a detection rate of approximately 0.001%. While this result in itself is informative, as it reflects how little overtly suspicious traffic was present in the live captures, it renders the Suricata-labeled dataset effectively unusable as a balanced evaluation target: any classifier predicting benign for all inputs would achieve 99.999% accuracy, making the signal indistinguishable from noise. Results on this variant are therefore reported for completeness and methodological transparency rather than as a primary evaluation target.

4.4 Botnet-Injected Variant

The third variant introduces a *synthetic attack component*. A controlled DDoS experiment was conducted on a dedicated server provisioned with dual AMD EPYC 7601 processors (100 logical cores) and 630 GB of RAM. Using Vagrant and Docker, 88 virtual machines were deployed on Debian 12: 86 attacker nodes (*minions*), each allocated one CPU core and 512 MB of RAM, a command-and-control master node, and a target services node running FTP, Telnet, SMTP, DNS, HTTP and POP3. The minions were orchestrated using a custom Python-based remote access tool developed for this purpose [24], consisting of a command server (`server.py`), response collector (`response.py`) and agent (`agent.py`) deployed on each minion, and directed to execute multi-vector DDoS attacks using MHDDoS [25], an open-source DDoS stress-testing framework. The attack was structured as a sequence of rounds executed by a coordinating shell script (`attacks.sh`) dispatched to all 86 minions simultaneously. Each round selected a random target port from the set {20, 23, 25, 53, 80, 8080}, a random thread count from 1 to 3, and a random attack duration from 10 to 30 seconds. Three attack protocols were cycled in sequence: TCP flood, UDP flood and SYN flood, with four rounds executed per protocol, producing twelve attack rounds in total. An inter-round sleep of twice the attack duration was added between the rounds to allow `tcpdump` to fully capture all the traffic burst before the next round began. The resulting captures totalled approximately 60 GB of raw attack traffic across all minions. From this experiment, the statistical characteristics of the attack traffic were used to generate 400,000 flows modelling the observed C2 botnet pattern (80 source nodes communicating in coordinated bursts toward a single target), which were then

merged with the benign baseline. The resulting dataset contains 1,687,049 flows with 23.71% malicious content, providing a class distribution sufficient for meaningful precision and recall evaluation. This variant constitutes the primary evaluation target for the cross-domain experiments reported in Chapter 5.

4.5 CIC Supplementary Dataset

In addition to the real-world dataset variants described above, a supplementary evaluation target was used for the three CIC-family implementations (`deeplearning_ids_fastai`, `deeplearning_ids_keras_tf` and `cic_ddos2019_deep_learning`). These models were trained on CIC-IDS2018 and CIC-DDoS2019 datasets and require a CIC-compatible feature schema for cross-domain evaluation. To assess their generalisation within the CIC dataset family before exposing them to real-world traffic, a community-assembled collection of CIC datasets was used [26]. This collection merges four datasets published by the Canadian Institute for Cybersecurity: CIC-IDS2017, CIC-DoS2017, CSE-CIC-IDS2018 and CIC-DDoS2019, into a single parquet file with harmonised labeling and removal of features identified as flawed in the literature, while preserving the original CICFlowMeter feature schema common to all four. The consolidated attack categories include DDoS, DoS, Botnet, Brute-force, Infiltration, Web attack and Port scan traffic. This dataset is not a real-world capture and carries the same caveats as any controlled benchmark; its role here is to establish whether the CIC-trained models generalise across the broader CIC family before the more demanding transfer to operational traffic is assessed in Chapter 5.

Taken together, the three labeling strategies expose a fundamental challenge that any real-world NIDS evaluation must confront: ground truth is expensive to obtain and rarely clean. The heuristic variant provides broad coverage but no verified attack signal, making it useful as a stress test rather than a ground-truth benchmark. The Suricata-based variant reveals that even a mature signature-based IDS found virtually no malicious activity in the live traffic, a result that is itself informative — it confirms that the benign baseline is genuinely representative of clean operational traffic, and not contaminated by undetected attacks that could distort the evaluation. The botnet-injected variant trades labeling realism for ground-truth certainty, providing the only partition where precision and recall can be computed reliably. Each variant is therefore reported in Chapter 5 with the interpretation appropriate to its labeling method, and the botnet-injected dataset is used as the primary evaluation target for quantitative comparison across models.

5 Cross-domain evaluation

5.1 Evaluation Methodology

This chapter presents the results of evaluating all thirteen implemented models against the real-world dataset variants described in Chapter 4. Each model was applied to data it had never seen during training, drawn from a different environment, with a different traffic distribution and, in most cases, a different labeling scheme than the one used in its source publication. The evaluation is structured around three distinct targets: the heuristic-labeled baseline, which provides broad coverage but no verified attack ground truth, then the Suricata-labeled variant, which is reported for methodological completeness despite its near-zero malicious sample count and the botnet-injected dataset, which constitutes the primary evaluation target as the only variant with verified malicious flows and a class distribution sufficient for meaningful precision and recall analysis. CIC-family models are additionally evaluated against the CIC collection benchmark to assess generalisation within their own dataset family before exposure to real-world traffic. One model `ddos.traffic.research.notebooks`, required a separate evaluation approach: its pre-saved FastAI learner files are not directly portable across environments, so inference was performed by reconstructing the model architecture from the save weights and mapping available features to the training schema. Due to feature mismatch between the CIC-DDoS2019 training data and the real-world parquets, only a subset of the model's input features could be populated, so the results for this model are therefore interpreted with that limitation in mind.

All models were evaluated using binary classification metrics: accuracy, weighted precision, weighted recall and weighted F1-score. Since the majority of models were trained on multi-class datasets (distinguishing between specific attack types such as DoS, Probe, R2L and U2R in NSL-KDD, or individual DDoS subtypes in the CIC family), their multi-class predictions were mapped to a binary format for cross-domain evaluation, meaning, any prediction corresponding to a non-benign class was treated as a positive detection, while predictions of the normal or benign class were treated as negative. This mapping was applied uniformly across all models via `label_mapper.py`, ensuring that the evaluation reflects detection capability in the most general sense,

whether the model flags traffic as suspicious, rather than its ability to identify specific attack subtypes that may not be present in the real-world captures. Weighted averages were used rather than macro averages to account for the class imbalance present in the heuristic and real-world datasets, where benign flows constitute 76% to 95% of the total sample depending on the variant. One consequence of this imbalance is that accuracy alone is an unreliable indicator of genuine detection performance: a model that predicts benign for every input achieves accuracy equal to the benign proportion of the dataset, which ranges from 0.76 on the botnet-injected variant to 0.95 on the heuristic baseline. Per-class precision and recall are therefore reported alongside aggregate metrics throughout this chapter and any model exhibiting near-zero malicious recall is explicitly identified as having undergone majority-class collapse regardless of its reported accuracy figure.

5.2 Overview of Results

Table 5: Cross-domain evaluation results per model and dataset variant

Model	Family	Heuristic	Botnet	Suricata	CIC Coll.
deeplearning_ids_fastai	CIC	0.9497	0.7629	0.9999	0.8538
deeplearning_ids_keras_tf	CIC	0.9497	0.7629	0.9999	0.7536
cic_ddos2019_deep_learning	CIC	0.9497	0.7629	0.9999	0.6122
ddos_traffic_research_notebooks ^a	CIC	0.2507	0.2319	—	0.5269
intrusion_detection_kdd_nsl	NSL	0.8188	0.2374	0.9689	—
network_intrusion_detection_bulk	NSL	0.8539	0.2486	—	—
decisiontree_ids	NSL	0.3564	0.4084	—	—
cbad_clustering	NSL	0.6122	0.4917	0.9390	—
nids_python_eval_rf	NSL	0.3594	0.4984	0.3196	—
nids_python_eval_svm	NSL	0.8539	0.2486	—	—
nids_python_eval_xgboost	NSL	0.9497	0.7629	—	—
nsl_gru_runner	NSL	0.8486	0.6818	—	—
anomaly_detection_kdd99_cnnlstm	NSL	0.6347	0.4747	—	—
<i>Majority-class baseline (benign proportion)</i>		0.9503	0.7629	0.9999	0.7822

^a Evaluated using reconstructed architecture from saved weights; feature coverage limited to available overlap with training schema. Only 15 of 68 input features available from real-world parquets.

^b Suricata variant contains only 16 malicious flows out of 1,287,049 total. Any model achieving accuracy near 0.9999 on this variant is trivially predicting benign for all inputs and the result carries nodiagnostic value. Results are included for completeness.

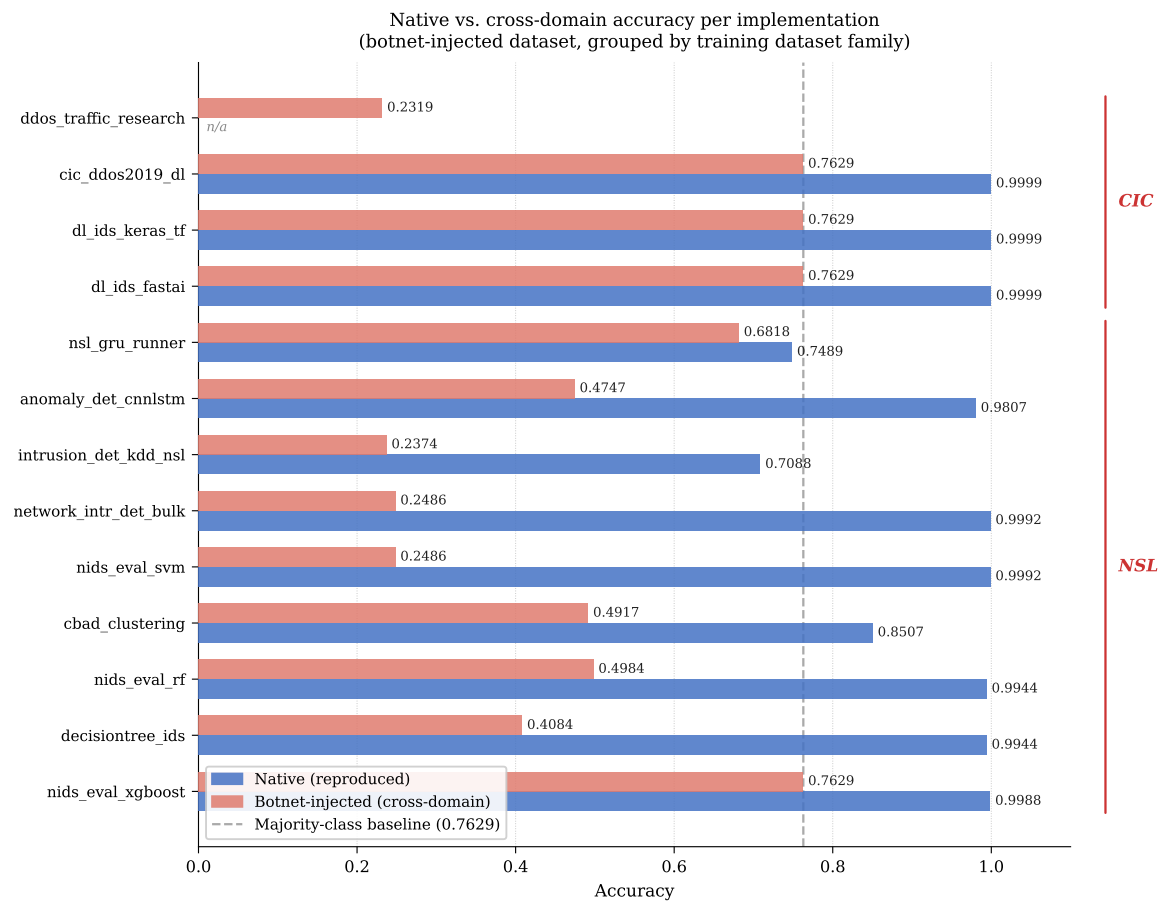


Figure 3: Native (reproduced) vs. cross-domain accuracy per implementation on the botnet-injected dataset

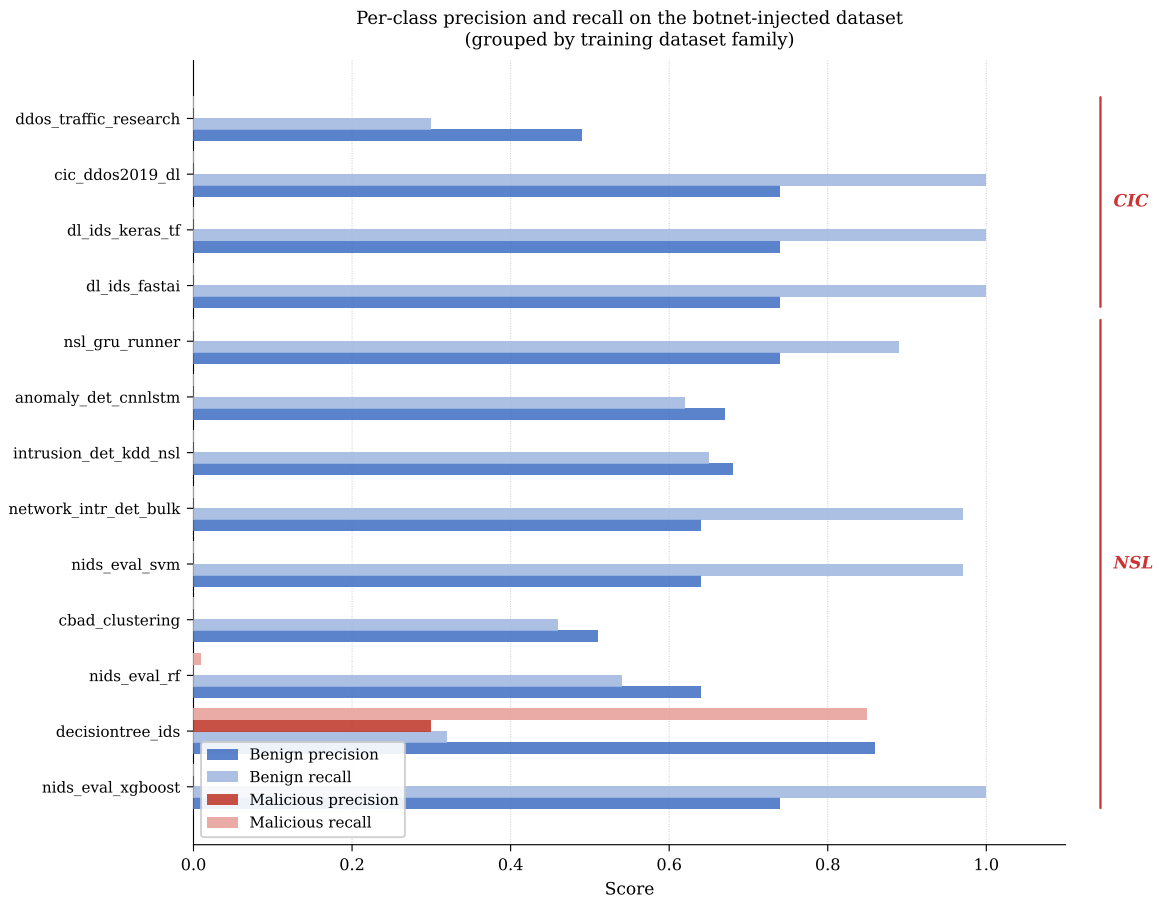


Figure 4: Per-class precision and recall on the botnet-injected dataset, grouped by training dataset family

Table 5 reveals a consistent and striking pattern across all thirteen models: performance on the botnet-injected dataset, the only evaluation target with verified ground-truth malicious flows, is substantially lower than native benchmark accuracy for every model without exception. The majority-class baseline row makes the scale of this problem explicit, a trivial classifier that predicts benign for every input achieves 0.7629 on the botnet variant simply by reflecting the dataset’s class distribution. Against this baseline, only **nsl_gru_runner** at 0.6818 exceeds it in any meaningful sense, while several models match it exactly, which just confirms complete majority-class collapse. The remaining models are at or below the trivial baseline, with several dropping well below 0.50, meaning that on the primary evaluation target, most of the models perform worse than just predicting benign for every input, meaning they are not merely failing to detect attacks, they are actively misclassifying benign traffic in sufficient volume to drag accuracy below the majority-class floor.

5.3 CIC-Family Models

The three CIC-trained models, `deeplearning_ids_fastai`, `deeplearning_ids_keras_tf` and `cic_ddos2019_deep_learning`, all produce identical accuracy figures across the real-world evaluation targets: 0.9497 on the heuristic dataset and 0.7629 on the botnet-injected variant. These values match the percentage of benign packets in each dataset exactly, confirming that all three models have undergone complete majority-class collapse and are predicting benign for every input. The apparent 0.9497 accuracy on the heuristic variant is therefore not a detection result, but it is actually a reflection of the fact that 94.97% of flows in that dataset are labeled benign, and a model that never fires produces this figure automatically. The same pattern holds on the botnet variant where the 76.29% benign proportion produces the 0.7629 accuracy.

On the CIC collection, which uses CIC-compatible features and therefore represents the most favourable cross-domain scenario for these models, performance diverges: `deeplearning_ids_fastai` achieves 0.8538, `deeplearning_ids_keras_tf` achieves a 0.7536, and `cic_ddos2019_deep_learning` drops to 0.6122. Even here, per-class inspection reveals that the higher accuracy figures are driven primarily by benign recall rather than genuine attack detection, with malicious recall at or near zero for all three models. The CIC collection result for `cic_ddos2019_deep_learning` is particularly notable, because despite being trained on a CIC-family dataset and evaluated on another CIC-family benchmark using the same feature schema, the model fails to generalise, suggesting that the patterns it learned are specific to the DDoS subtypes and traffic conditions present in CIC-DDoS2019 rather than to any general notion of malicious network behaviour.

`ddos_traffic_research_notebooks` was evaluated separately using a reconstructed architecture from its pre-save weights, with only 15 of 68 features available from the real-world parquets. On the heuristic and botnet variants it achieves a 0.2507 and 0.2319 respectively, substantially below the majority-class baseline, indicating that the partial feature vector and the mismatch between the BatchNorm normalisation statistics embedded during training and the unnormalised real-world inputs produce predictions that are worse than trivial. On the CIC collection, where 57 of 68 features could be populated, accuracy rises to 0.5269, still well below the majority-class baseline. This result illustrates a broader portability problem with pre-saved FastAI learner files: the preprocessing statistics are baked into the model at training time and cannot be transferred to a new feature distribution without retraining, making genuine cross-domain evaluation infeasible even when the feature schema nominally matches.

5.4 NSL-KDD-Family Models

The ten models trained on NSL-KDD or KDD99 show considerably more variation in cross-domain performance than the CIC family, though the dominant pattern remains majority-class collapse on the botnet-injected dataset. A structural factor common to all NSL-KDD evaluations must be noted upfront: as described in Chapter 4, only 11 of the 41 NSL-KDD features could be derived from passive flow captures, with the remaining 30 zero-padded. Every result in this section is therefore obtained under a partial feature vector, and the performance figures should be interpreted as a lower bound on what these models might achieve with complete feature coverage rather than a direct reflection of their architectural capability.

On the heuristic dataset, results split into two clusters. Five models (`network_intrusion_detection_bulk`, `nids_python_eval_svm`, `ns1_gru_runner`, `intrusion_detection_kdd_ns1` and `anomaly_detection_kdd99_cnnlstm`) achieve accuracy between 0.63 and 0.87, while the remaining five fall below 0.62. However, as established earlier heuristic accuracy figures are unreliable as detection metrics because the labels themselves are derived from the same flow statistics that these models use as features. A model that learned to associate high packet rates with attacks will naturally agree with a labeling scheme that also flags high packet rates, producing inflated accuracy without any genuine detection signal.

On the botnet-injected dataset, the picture becomes clearer. The majority of NSL-KDD models collapse to or below the 0.7629 majority-class baseline. `ns1_gru_runner` is the only model to remain meaningfully above baseline at 0.6818, though this is still a substantial drop from its reproduced native accuracy of 0.7489 on KDDTest+. The faithfully replicated models (`ns1_gru_runner`, `intrusion_detection_kdd_ns1` and `anomaly_detection_kdd99_cnnlstm`) are particularly informative here because their native performance is known precisely from the replication runs in Chapter 3.

`ns1_gru_runner` drops from 0.7489 to 0.6818, `intrusion_detection_kdd_ns1` from 0.7088 to 0.2374 and `anomaly_detection_kdd99_cnnlstm` from 0.9807 to 0.4747, and this last model's collapse is especially worth mentioning, as a model that reproduced its paper results almost exactly on KDD99 falls to below the coin-flip threshold when exposed to real network traffic with injected botnet flows.

Three models warrant individual discussion. `decisiontree_ids` achieves only 0.3564 on the heuristic dataset and 0.4084 on the botnet variant, both well below the majority-class baseline. Inspection of per-class metrics shows the model predicts malicious for the majority of inputs, the inverse of the majority-class collapse seen elsewhere. This suggests that the decision tree learned a narrow set of feature thresholds that happen to fire aggressively on real-world traffic distributions, producing high false positive rates rather than zero detection. `cbad_clustering` behaves similarly, achieving 0.6122 on heuristic and 0.4917 on botnet with imbalanced per-class predictions. As a K-Means clustering model, it assigns class membership based on distance to centroids learned from NSL-KDD, and the real-world feature distribution places many flows near the attack centroids despite them being benign. `nids_python_eval_rf` at 0.3594 on heuristic and 0.4984 on botnet also falls below baseline, though its botnet result is the closest of any non-collapsing model to the majority-class floor, suggesting partial detection of some botnet features while still misclassifying a substantial portion of benign traffic.

Across the NSL-KDD family as a whole, the results confirm that benchmark performance provides no reliable signal about real-world detection capability. Models that reproduced their native results closely, such as `anomaly_detection_kdd99_cnnlstm` at 0.9807, perform no better and in some cases substantially worse than models with more modest native results. The feature space mismatch introduced by zero-padding 30 of 41 features is a contributing factor, but it cannot explain the full scale of the degradation: even the features that are available, such as duration, packet counts, byte counts and flag counts, are present in both training and evaluation data, yet the models fail to generalise from their benchmark distributions to live traffic.

5.5 Summary

The cross-domain evaluation presented in this chapter produces a result that is both clear and consistent: none of the thirteen evaluated models demonstrates reliable intrusion detection capability when applied to traffic outside its native benchmark environment. On the botnet-injected dataset, which provides the only evaluation target with verified ground-truth malicious flows and a class distribution sufficient for meaningful measurement, eleven of thirteen models perform at or below the majority-class baseline of 0.7629. The two that exceed it do so by narrow margins and with near-zero malicious recall, meaning their accuracy advantage over a trivial always-benign classifier reflects benign classification performance rather than genuine attack detection.

The failure modes observed across the evaluation fall into two broad categories. The

first and most common is majority-class collapse: the model predicts benign for essentially all inputs, achieving accuracy equal to the benign proportion of the dataset while contributing nothing to security. This pattern was observed across all three CIC-family models on real-world targets, across several NSL-KDD models, and was confirmed by per-class recall figures of zero or near-zero for the malicious class. The second failure mode, observed in `decisiontree_ids`, `cbad_clustering` and `nids_python_eval_rf`, is inverse collapse: the model predicts malicious too aggressively, accumulating false positives on benign real-world traffic to the point where accuracy falls below the majority-class floor. Both failure modes share a common cause: the statistical patterns these models learned from their benchmark datasets do not transfer to the real-world traffic distribution, in either direction.

Several structural factors amplify this generalisation gap. The 30-feature zero-padding applied to all NSL-KDD evaluations removes content and host-based statistics that these models rely on during benchmark evaluation, placing them at a systematic disadvantage relative to their native conditions. The heuristic labeling scheme introduces a circularity problem in which models that learned packet-rate-based patterns artificially agree with packet-rate-based pseudo-labels, inflating accuracy without providing detection signal. The Suricata-labeled variant confirmed that the underlying live traffic contains virtually no overtly suspicious flows, validating the benign baseline but rendering that variant useless as an evaluation target. These are not incidental limitations of this particular evaluation, instead they reflect inherent challenges in any attempt to assess benchmark-trained IDS models against operational network traffic, and they point to a structural gap between how models are evaluated in the literature and how they would need to perform in practice.

The implications of these finding for the broader field of ML-NIDS research are discussed in Chapter 6.

6 Implications

The findings presented in Chapter 5 carry implications that extend beyond the specific models and datasets examined in this evaluation. The consistent failure of the evaluated ML-NIDS models to generalise to real-world traffic is not a coincidence of this experimental setup, nor is it an artifact of the specific real-world dataset chosen. It reflects a structural problem in how intrusion detection research has been conducted for more than two decades. The benchmarks used to measure progress have become decoupled from the environments in which detection systems must actually operate. This chapter discusses what those findings mean in practice, first for the research community, then for the security experts who must decide whether to trust and deploy models like these, and lastly for the broader question of how the field should be moving forward, if it wants to produce detection systems that are actually useful outside of a laboratory context.

The most direct implication of this work concerns the validity of benchmark-based evaluation as the primary means of measuring progress in ML-NIDS research. The thirteen models evaluated in this thesis were selected precisely because they represented a broad cross-section of the field (different architectures, different datasets, different publication years and different levels of reported performance). Despite this diversity, their behavior on real-world traffic was remarkably uniform. The near-universal collapse on majority-class prediction on the botnet-injected dataset suggests that what these models have learned is not a general representation of malicious network behavior, but rather a set of statistical associations that happen to be predictive within the narrow distributional boundaries of their training benchmarks. When those boundaries are crossed, the learned associations no longer hold and the models have no fallback beyond predicting the dominant class.

This has a direct consequence for how results in the literature should be interpreted. A reported accuracy of 0.99 on NSL-KDD does not mean that a model can detect 99% of intrusions on a live network. It means that the model has learned to separate the specific statistical distributions of NSL-KDD's training and test partitions with high fidelity. These are not the same thing and the field has largely treated them as if they

were. The reproducibility evaluation in Chapter 3 reinforces this point: even models whose reported figures could be closely reproduced, which confirms that the implementations were correct and the experiments repeatable, collapsed on real-world data. Reproducibility of a benchmark result is therefore a necessary but far from sufficient condition for claiming that a model works. The fact that a result can be replicated does not make it meaningful.

A subtler but equally important implication concerns the choice of evaluation metric. Accuracy is the primary reported figure in the majority of ML-NIDS papers, and on balanced or near-balanced benchmark datasets it is a reasonable summary statistic. However, as this evaluation demonstrates, on datasets with realistic class imbalance, accuracy is easily manipulated by majority-class collapse. A model that achieves 0.9999 accuracy on a dataset that is 99.99% benign simply predicts the majority class. The field's reliance on accuracy as the main figure, particularly on datasets that have been criticized for unrealistic class distribution, has allowed this failure mode to go largely unnoticed in published evaluations. Precision and recall on the minority (malicious) class are far more informative metrics for intrusion detection, and their systematic absence from many papers in this area is a methodological gap that has contributed to the inflation of reported results.

For security experts and organizations considering the deployment of ML-based intrusion detection, the findings of this evaluation suggest a straightforward but uncomfortable conclusion, which is that the published accuracy figures for the examined models should not be taken as evidence of operational readiness. A model that achieves 0.99 accuracy on NSL-KDD and 0.47 on real-world traffic with injected botnet flows is not a model that is 99% effective at detecting intrusions, but instead, it is a model that has been measured in a context that does not resemble deployment conditions. The gap between those two numbers is not a minor discrepancy to be acknowledged as a limitation; it actually represents the difference between a system that provides security value and one that provides false confidence.

This matters particularly because the decision to adopt or reject an ML-based detection system is rarely made by the researchers who built it. It is made by security engineers, procurement teams, and operations staff who rely on published benchmarks as proxies for real-world performance because they do not have the resources to conduct independent evaluations at scale. When those benchmarks systematically overstate performance, not through fraud or error but through the structural limitation of being measured on data that does not represent production traffic, the result is a

misalignment between what practitioners believe they are getting and what they are actually deploying. The models evaluated would, if deployed on the network from which the evaluation data was capture, either miss almost all malicious traffic or generate false positive rates high enough to render the system operationally unusable.

A related concern is the invisibility of failure. Majority-class collapse is a particularly dangerous in a deployment context because it is silent. A model that predicts benign for every input does not produce alerts, does not trigger investigations, and does not generate any visible signal that something has gone wrong. From the perspective of a monitoring dashboard, it looks identical to a model that is correctly determining that the current traffic is clean. Without ground-truth labeling of live traffic there is no straightforward way for an operator to distinguish between a correctly functioning model and one that has collapsed. This makes benchmark-trained models not merely ineffective but actively misleading in operational settings, because their failure mode produces no observable signal that they have failed.

Addressing the generalization gap found in this evaluation requires changes at multiple levels of the research process, not merely the addition of a real-world evaluation step at the end of an unchanged pipeline. The most fundamental change needed is in how benchmark performance is framed in published work. Reporting accuracy on a single benchmark dataset as the primary evaluation metric of a model's capability should be treated as insufficient. The field would benefit from adopting evaluation norms that require, at minimum, cross-dataset testing within the same domain family before any generalization claims are made, and ideally out-of-distribution evaluation on data that differs meaningfully from the training environment in terms of traffic composition, time period or network topology.

Dataset design is an equally important layer. The dominance of NSL-KDD and the CIC family in this evaluation reflects the dominance of those datasets in the literature, and that dominance is itself part of the problem. When the entire field converges on a small number of benchmarks, models optimize for those benchmarks rather than for the underlying detection task. Introducing new datasets that reflect diverse network environments, that include novel attack patterns, and that are periodically updated to reflect the current state of both benign and malicious traffic would reduce the incentive to overfit to a fixed distribution. The construction methodology used in Chapter 4 of this thesis represents one possible template for how evaluation datasets might be constructed with greater attention to operational realism, even if the resulting dataset is imperfect by any standard.

Finally, the results of this evaluation suggest that the architectural diversity visible in the literature has not translated into diversity of generalization behavior. All architectures collapsed under the same conditions, which implies that the bottleneck is not model complexity or architecture choice, but instead, the distributional gap between training and testing data. Techniques from the broader machine learning literature that are designed to address this gap, including domain adaptation, distribution shift detection and uncertainty quantification, have seen limited adoption in the ML-NIDS field. Incorporation of these approaches together with more rigorous evaluation methodology, would offer a more promising path toward detection systems that retain meaningful capability outside their training distribution that continued refinement of architectures tested exclusively on fixed benchmarks.

7 Conclusion

This thesis set out to investigate whether the high classification accuracy reported by open-source ML-NIDS implementations on standard benchmark datasets reflects genuine intrusion detection capability, or whether it is an artifact of training and evaluation conditions that do not represent operational network traffic. To answer that question, thirteen open-source implementations with different architectures, training datasets and publication years were first re-evaluated on their native datasets to establish a reproducibility baseline, then evaluated against a real-world dataset constructed from live network captures collected in April 2026. The real-world dataset was produced with three labeling variants (heuristic, Suricata-based and botnet-injected) and in two feature schemas to enable consistent cross-model comparison across both NSL-KDD-trained and CIC-trained models. The results were indisputable: on the botnet-injected dataset, which provided the only evaluation target with verified ground-truth malicious flows, eleven of the thirteen models performed at or below the majority-class baseline, with most displaying complete majority-class collapse and near-zero malicious recall. No model demonstrated reliable detection capability on traffic it had not been trained on, regardless of its reported accuracy or its architecture.

Several limitations of this evaluation should be acknowledged. The real-world capture was obtained from a single server over three collection time frames in April 2026, which means the benign baseline reflects the traffic patterns of one specific environment at one specific time. A server with different application profiles, user behavior or geographic location would produce a different benign distribution, and models might generalize differently to it. The malicious traffic in the primary evaluation target was synthetically injected rather than observed in the wild, which means the attack patterns are those of a controlled DDoS experiment rather than the full diversity of real-world intrusion attempts. The feature mismatch introduced by zero-padding 30 of 41 NSL-KDD features is a structural limitation of the evaluation pipeline that places NSL-KDD models at a systematic disadvantage, and the true generalization gap for those models may be either larger or smaller than what was seen here, had complete feature coverage been achieved. These caveats do not invalidate the findings, as the collapse was consistent and dramatic enough that partial feature coverage alone cannot

explain it, but instead they do bound the claims that can be made on the basis of a single evaluation study.

Future work in this area would benefit from several directions. Expanding the real-world evaluation to multiple network environments and capture periods would strengthen the generalisability of the findings and allow patterns in which model families transfer better across different traffic types to be identified more precisely. Developing labeling methodologies that can produce verified ground truth from live traffic without relying on synthetic injection would address the most significant limitation of current real-world evaluation approaches. At the modelling level, the application of domain adaptation and distribution shift detection techniques to the ML-NIDS problem deserves systematic investigation. If models can be made aware that their input distribution has shifted from training conditions, they may be able to signal uncertainty rather than silently collapsing to majority-class prediction, which would at least make the failure mode visible. Likewise, the adoption of standardised out-of-distribution evaluation as a required component of ML-NIDS benchmarking, rather than an optional addition, would over time create pressure toward models that generalise rather than models that merely memorize their training distribution.

8 Bibliography

- [1] Mahbod Tavallaee, Ebrahim Bagheri, Wei Lu, and Ali A. Ghorbani. A detailed analysis of the KDD CUP 99 data set. In *Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, 2009. (Cited on pages 1, 3, 5 in 6.)
- [2] UCI Machine Learning Repository. KDD cup 1999 data, 1999. (Cited on page 1.)
- [3] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018. (Cited on page 1.)
- [4] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018. (Cited on page 1.)
- [5] Iman Sharafaldin, Arash Habibi Lashkari, Saqib Haghighat, and Ali A. Ghorbani. Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy. In *Proceedings of the 2019 International Carnahan Conference on Security Technology (ICCST)*, 2019. (Cited on page 1.)
- [6] Nour Moustafa and Jill Slay. UNSW-NB15: A comprehensive data set for network intrusion detection systems. In *Proceedings of the 2015 Military Communications and Information Systems Conference (MilCIS)*, 2015. (Cited on page 1.)
- [7] Quamar Niyaz, Weiqing Sun, Ahmad Y. Javaid, and Mansoor Alam. A deep learning approach for network intrusion detection system. In *Proceedings of the 5th International Conference on Security and Privacy in Communication Networks (EAI)*, 2015. (Cited on pages 4, 9 in 11.)
- [8] Herve Nkiamana, Syed Zainudeen Mohd Said, and Muhammad Saidu. A subset feature elimination mechanism for intrusion detection system. *International Journal of Advanced Computer Science and Applications*, 7(4), 2016. (Cited on pages 4 in 9.)

-
- [9] R. Vinayakumar, K. P. Soman, and Prabaharan Poornachandran. Applying convolutional neural network for network intrusion detection. In *Proceedings of the International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, 2017. (Cited on pages 4 in 9.)
 - [10] R. Vinayakumar, K. P. Soman, and Prabaharan Poornachandran. Evaluating effectiveness of shallow and deep networks to intrusion detection system. In *Proceedings of the International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, 2017. (Cited on pages 4 in 9.)
 - [11] R. Vinayakumar, K. P. Soman, and Prabaharan Poornachandran. Evaluation of recurrent neural network and its variants for intrusion detection system (IDS). *International Journal of Information System Modeling and Design*, 10(1), 2019. (Cited on pages 4 in 9.)
 - [12] Congyuan Xu, Jizhong Shen, Xin Du, and Fan Zhang. An intrusion detection system using a deep neural network with gated recurrent units. *IEEE Access*, 2018. (Cited on pages 4, 7 in 9.)
 - [13] Ram B. Basnet, Riad Shash, Clayton Johnson, Lucas Walgren, and Tenzin Doleck. Towards detecting and classifying network intrusion traffic using deep learning frameworks. *Journal of Internet Services and Information Security*, 9(4), 2019. (Cited on pages 4 in 9.)
 - [14] Jeremy Perez, Bethany Danner, and Veronika Neeley. Optimizing preprocessing & parameters for cluster-based anomaly detection. Technical report, Montana State University, National Science Foundation, August 2019. (Cited on pages 4 in 9.)
 - [15] Siddhant Agarwal, Abhay Tyagi, and G. Usha. A deep neural network strategy to distinguish and avoid cyber-attacks. In Subhransu Sekhar Dash, C. Lakshmi, Swagatam Das, and Bijaya Ketan Panigrahi, editors, *Artificial Intelligence and Evolutionary Computations in Engineering Systems*, pages 673–681, Singapore, 2020. Springer Singapore. (Cited on pages 4 in 9.)
 - [16] Marcos V. O. Assis, Luiz F. Carvalho, Jaime Lloret, and Mario L. Proença Jr. A GRU deep learning system against attacks in software defined networks. *Journal of Network and Computer Applications*, 177, 2021. (Cited on pages 4 in 9.)
 - [17] Benedetto Marco Serinelli, Anastasija Collen, and Niels Alexander Nijdam. On the analysis of open source datasets: Validating IDS implementation for well-known and zero day attack detection. *Procedia Computer Science*, 2021. (Cited on pages 5, 7 in 9.)

-
- [18] James Halladay, Drake Cullen, Nathan Briner, Jackson Warren, Karson Fye, Ram Basnet, Jeremy Bergen, and Tenzin Doleck. Detection and characterization of DDoS attacks using time-based features. *IEEE Access*, 2022. (*Cited on pages 5 in 9.*)
 - [19] Arash Habibi Lashkari. CICFlowMeter: Network traffic biflow generator and analyser, 2017. (*Cited on page 6.*)
 - [20] Gints Engelen, Vera Rimmer, and Wouter Joosen. Troubleshooting an intrusion detection dataset: The CICIDS2017 case study. In *2021 IEEE Security and Privacy Workshops (SPW)*, pages 7–12. IEEE, 2021. (*Cited on page 7.*)
 - [21] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé, and Eric Totel. Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes. In *Risks and Security of Internet and Systems (CRiSIS 2022)*, volume 13857 of *Lecture Notes in Computer Science*, pages 19–38. Springer, 2023. (*Cited on page 7.*)
 - [22] Philippe Biondi et al. Scapy: Packet crafting for python, 2023. (*Cited on page 16.*)
 - [23] Open Information Security Foundation. Suricata network threat detection engine, 2023. (*Cited on page 17.*)
 - [24] Danilo Lazić. dnllzc-rat: Custom remote administration tool, 2024. (*Cited on page 17.*)
 - [25] MatrixTM. MHDDoS: Multi-vector HTTP DDoS attack script, 2022. (*Cited on page 17.*)
 - [26] dhoogla. CIC-IDS collection: Combined Canadian Institute for Cybersecurity intrusion detection datasets. Kaggle, 2023. Harmonised collection of CIC-IDS2017, CIC-DoS2017, CSE-CIC-IDS2018, and CIC-DDoS2019. (*Cited on page 18.*)
 - [27] Markus Ring, Sarah Wunderlich, Deniz Scheuring, Dieter Landes, and Andreas Hotho. A survey of network-based intrusion detection data sets. *Computers & Security*, 86:147–167, 2019. (*Not cited.*)